

Промпт-инжиниринг и работа с кодом

AI-инструменты в разработке



Проверить, идет ли запись

Меня хорошо видно & слышно?





Андрей Сорокин

Ведущий разработчик .NET, преподаватель в OTUS

- В коммерческой разработке ПО с 2011 года
- Стэк: C#, ASP.NET, JS, TS, React, AI, LLM, ...
- Направления проектов: САПР, CRM, ЭДО, IoT, ...

 [@aprogrammerru](https://t.me/aprogrammerru)

 andrey@aprogrammer.ru

 [@aprogrammer-ru](https://github.com/aprogrammer-ru)

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе в
Telegram



Задаем вопрос
в чат или голосом,
так же, “поднимайте руку”



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Основы ИИ в разработке

ИИ в сопровождении и архитектуре

Расширенные техники и
кастомизация

Собственный проект

ФИНИШ



Маршрут вебинара

Введение в промптинг

Базовые шаблоны и техники

Как работает RAG

Практика с Cline, Continue

Рефлексия



Цели вебинара

К концу занятия вы сможете

Смысл

Для чего вам это уметь

формулировать эффективные
промпты для генерации,
анализа и отладки кода

Понимать принципы промптинга

Повышать качество ответов от LLM

Применять на практике



Введение в пром프트-инжиниринг

Что такое промпт?

1. Простой вопрос от человека к машине
2. Структурированный запрос под конкретную LLM
3. Алгоритм для работы с нейросетью
4. Секретное слово понятное только ИИ



Промпт

- Это точно сформулированный запрос или инструкция, адресованная языковой модели, которая определяет контекст, задачу и желаемый формат ответа.

Почему это важно?

- Качество промпта напрямую влияет на точность и полезность получаемого от ИИ результата.

Общие принципы эффективного промπτинга

- 1. Точность** - насколько точно ИИ понимает и выполняет поставленную задачу
 - Пофикси этот репозиторий" → ИИ не понимает, что именно нужно исправить
 - + "Исправь опечатки в файлах *.cs в директории ./WebApi"
- 2. Структурированность** - логическая организация запроса
 - длинный текст без абзацев, перечислений и структуры
 - + четкое разделение на: задачу, требования, ограничения, ожидаемый результат
- 3. Контекст** - понимание ИИ специфики проекта/приложения
 - "Реализуй алгоритм сортировки" → нет контекста, для какого стека технологий
 - + "Реализуй алгоритм сортировки пузырьком на языке Go"



Базовый шаблон эффективного промпта:

1. **Роль:** Кто ты?
2. **Задача:** Что сделать? (Конкретное действие)
3. **Контекст:** Какая информация нужна? (Технологии, входные данные, ограничения)
4. **Требования/Ограничения:** Как сделать? (Стиль, производительность, запреты)
5. **Цель/Результат:** Зачем это нужно? (Какой конечный продукт вы хотите увидеть)



Базовый шаблон эффективного промпта:

Роль: Ты — senior Python-разработчик, специализирующийся на чистом и эффективном коде.

Задача: Напиши функцию `calculate(expression)` для вычисления простых математических выражений.

Контекст:

- Язык: Python 3.10+
- Входные данные: строка `expression` (напр., `"2 + 3 * 4"`).
- Нужно поддерживать операции: `+`, `-`, `*`, `/` и скобки `()`.

Требования:

- Используй стандартную библиотеку. Сторонние библиотеки запрещены.
- Функция должна обрабатывать ошибки (неверный синтаксис, деление на ноль).
- Код должен быть хорошо документирован (docstring, комментарии).
- Учитывай приоритет операций и скобки.

Цель: Я хочу использовать эту функцию в высоконагруженном проекте, поэтому важна читаемость и надежность.



Фундаментальные техники

- ✓ Zero-Shot (без примеров)
- ✓ One-Shot / Few-Shot (с примерами)
- ✓ Chain-of-Thought (цепочка рассуждений)

Zero-Shot промптинг (без примеров)

Суть: Вы просите модель выполнить задачу, не предоставляя никаких примеров.

Как это работает: Вы полагаетесь на обширные знания, полученные моделью во время обучения. Например, запрос "Напиши React-компонент для модального окна" сработает, потому что модель знает, что такое React и как выглядят типичные модальные окна.

Когда использовать: Для стандартных, хорошо известных задач и шаблонов разработки. Это самый простой и быстрый вариант запроса.

Не подходит: Для уникальных или узкоспециализированных проектов с закрытой кодовой базой и документацией.



One-Shot / Few-Shot (с примерами)

Суть: Вы предоставляете модели один или несколько примеров того, как должна быть выполнена задача, чтобы задать шаблон ожидаемого ответа.

Как это работает: Полагаемся не только на встроенные знания модели. Показываем ей конкретный стиль кода, соглашения об именовании, подходы, паттерны. Пример: «Вот как у нас написан endpoint для получения пользователей: [код...]. Задача: Теперь напиши похожий endpoint для создания пользователя».

Когда использовать: Для уникальных или узкоспециализированных проектов с закрытой кодовой базой и документацией.

Особенности: При использовании ИИ агентов/ассистентов, инструмент разработки автоматически добавляет примеры кода из кодовой базы к вашим запросам.



Chain-of-Thought (цепочка рассуждений)

Суть: Вы явно описываете логические шаги, которые ИИ должен предпринять для решения сложной задачи.

Как это работает: Вместо запроса о конечном результате, вы проводите агента через процесс по шагам. «Эта функция падает с ошибкой `TypeError`. Сначала проанализируй лог сборки на наличие предупреждений. Затем проверь документацию функции `n`, которую она вызывает. После этого найди в коде все места, где может передаваться неверный тип аргумента. Наконец, предложи исправление».

Когда использовать: При отладке сложных неоднозначных ситуаций, требующих детального разбора по шагам. Направляет рассуждения модели по заданному маршруту, помогает избежать поспешных и, возможно, неверных решений, галлюцинаций, повышает точность локализации сложных проблем



Способы оценки качества промпта

- **A/B тестирование** - сравнение результатов использования разных формулировок промптов
- **Оценка по метрикам** - измерение времени ответа, количества уточняющих вопросов, точности ответа
- **Повторяемость** - проверка стабильности результатов при многократном использовании одного промпта
- **Обратная связь** - оценка удовлетворенности пользователя качеством ответа ИИ



Вопросы



если есть вопросы



если вопросов нет

LIVE



Retrieval-Augmented Generation

Что такое RAG

RAG (Retrieval-Augmented Generation) — это продвинутая архитектура, которая позволяет ИИ-модели использовать релевантную информацию из вашей собственной базы знаний (документация, кодовая база, внутренние wiki) для генерации точных и контекстуальных ответов.

Простая аналогия: Представьте, что вы задаёте вопрос юристу. Без RAG он отвечает только на основе своих общих знаний. С RAG — он сначала ищет в ваших конкретных договорах и законах, а затем даёт ответ, подкреплённый найденными документами.



Как работает RAG?

1. Запрос (**Query**): Вы задаёте вопрос (например, «Как в нашем проекте настроена аутентификация?»)
2. Поиск (**Retrieval**): Агент ищет упоминания об аутентификации в:
 - Текущих открытых файлах кода.
 - Всей структуре вашего проекта
 - Во внешних системах (browser/MCP)
 - Документах технической спецификации (`.md`-файлы).
3. Расширение контекста (**Augmentation**): Найденные релевантные фрагменты кода и текста добавляются к вашему исходному промпту.
4. Генерация (**Generation**): Модель создаёт окончательный ответ, основанный на найденной информации, что обеспечивает высокую точность и соответствие вашему проекту.



LIVE



Подведём итоги занятия

Итоги

- Всё ли получилось?
- Какие проблемы были?

Вопросы



если есть вопросы



если вопросов нет

Полезные материалы

Руководства по промптингу

<https://openai-docs.ru/docs/guides/prompt-engineering>

<https://www.promptingguide.ai/applications/coding>

<https://habr.com/ru/articles/834054/>

Базы данных шаблонов промптов

<https://alice.yandex.ru/prompthub/>

<https://platform.claude.com/docs/en/resources/prompt-library/library>

Плагины для работы с LLM

<https://docs.cline.bot/introduction/welcome>

<https://docs.continue.dev/>

API платформы с OpenSource LLM

<https://openrouter.ai/>

<https://console.groq.com/keys>



**You are our
OTUS heroes**

